FLUID

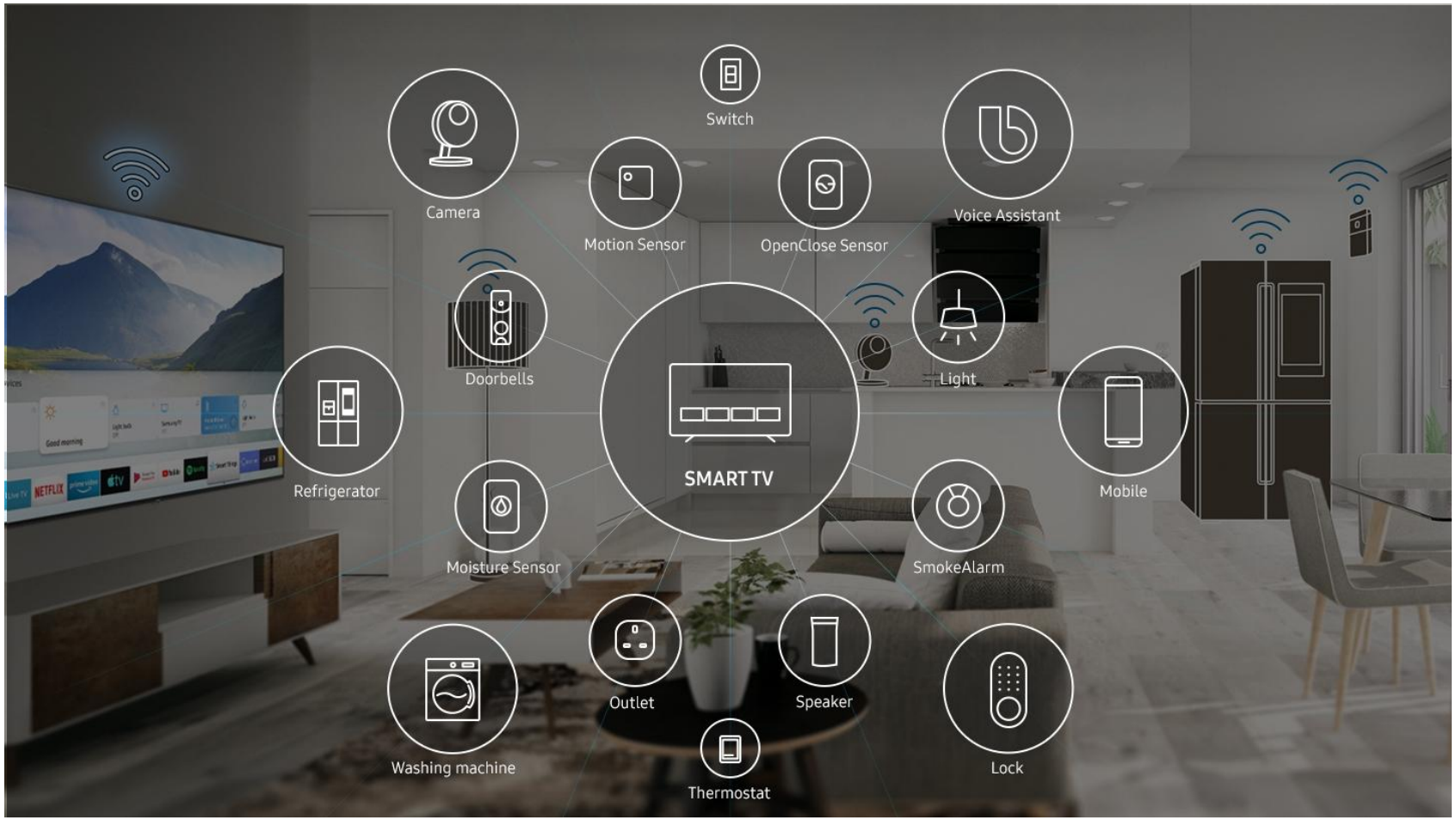
Enabling next-generation multi-device paradigm

새로운 멀티-디바이스 패러다임을 위한 차세대 모바일 플랫폼

Insik Shin
KAIST & Fluiz

Many Many Mobile (Smart) Devices

- Many smart devices are around us
  - smartphone, tablet, watch, TV, home appliance, car etc.

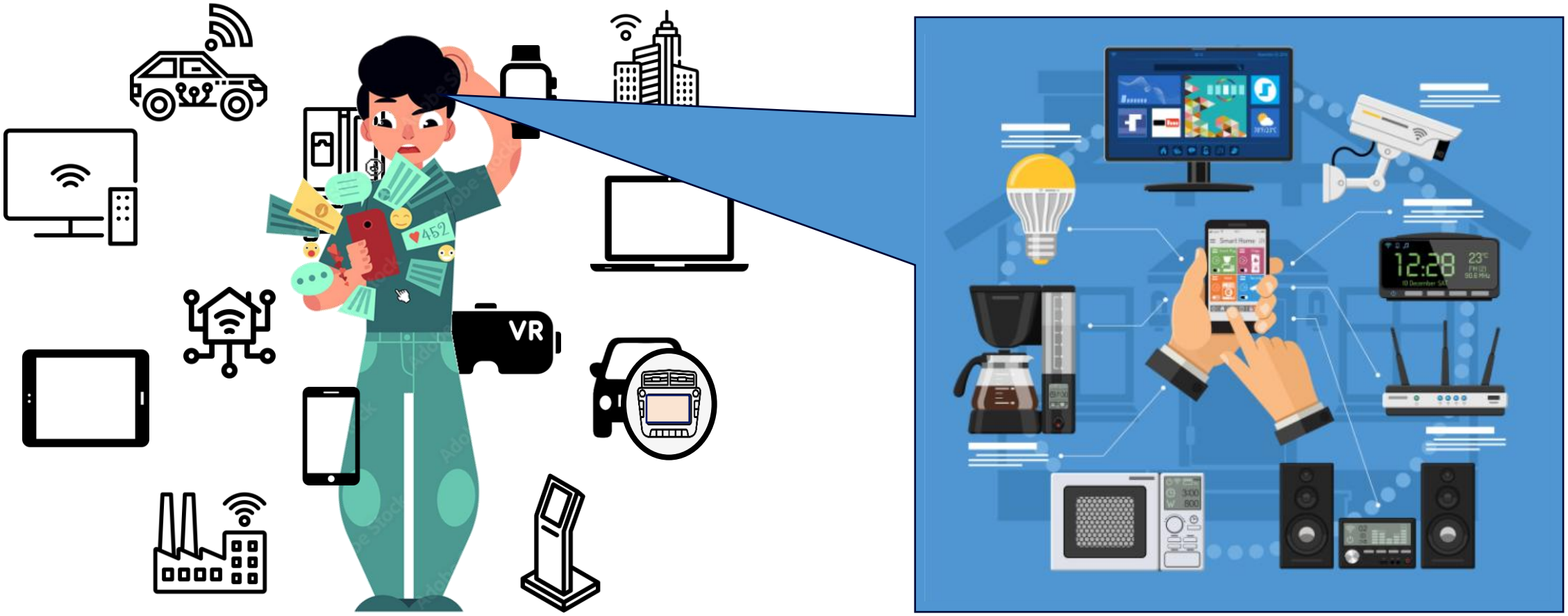


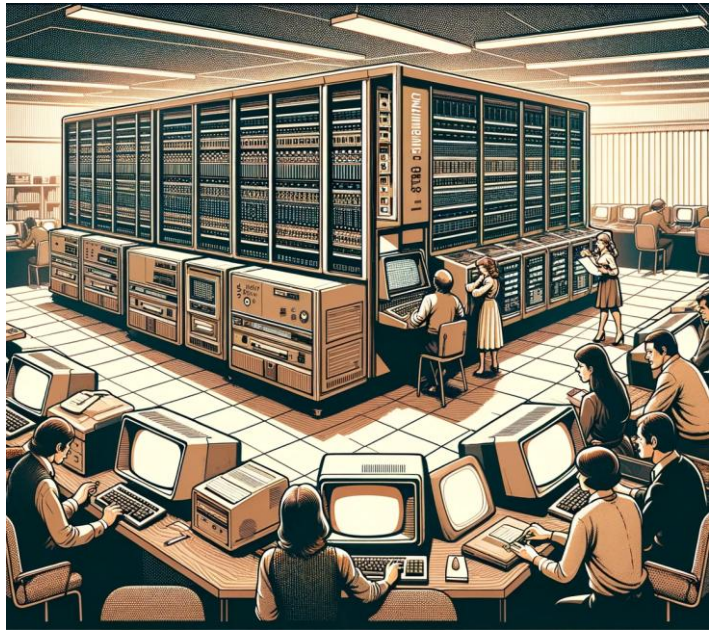
pictures from samsung.com

Many Many Mobile (Smart) Devices

Users naturally seek to use multiple devices simultaneously

even for unprecedented UX





Paradigm Shifts

Any ideas for

new abstractions (computing models)

for multi-device experience (environments)?

Example: Car Racing Game (single-device app)

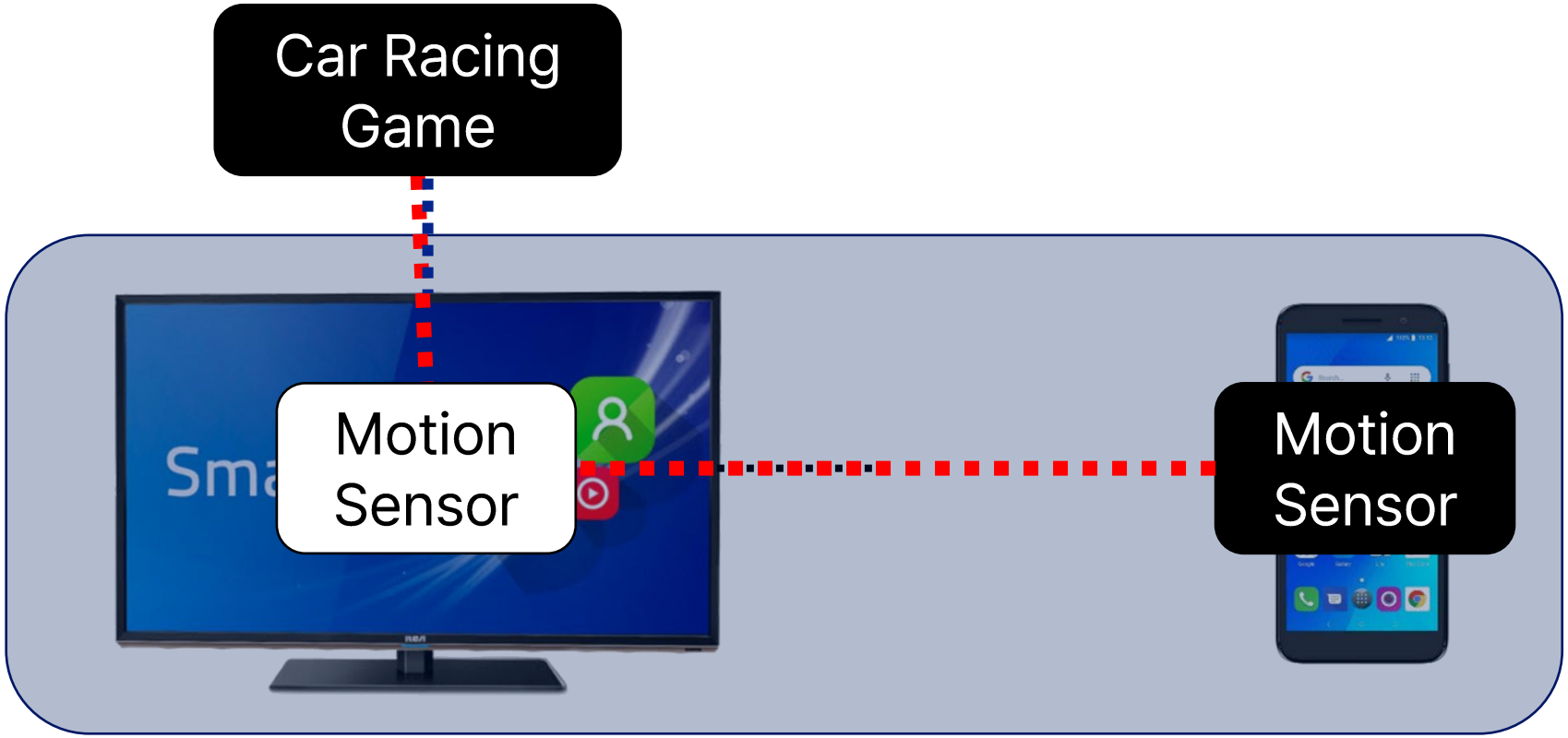
- Car racing game: smartphone mobile app

Example: Car Racing Game (single-device app)

- Car racing game: smartphone mobile app
- **Suppose a user wants to run the game app on a large screen device (TV) for a more immersive experience**

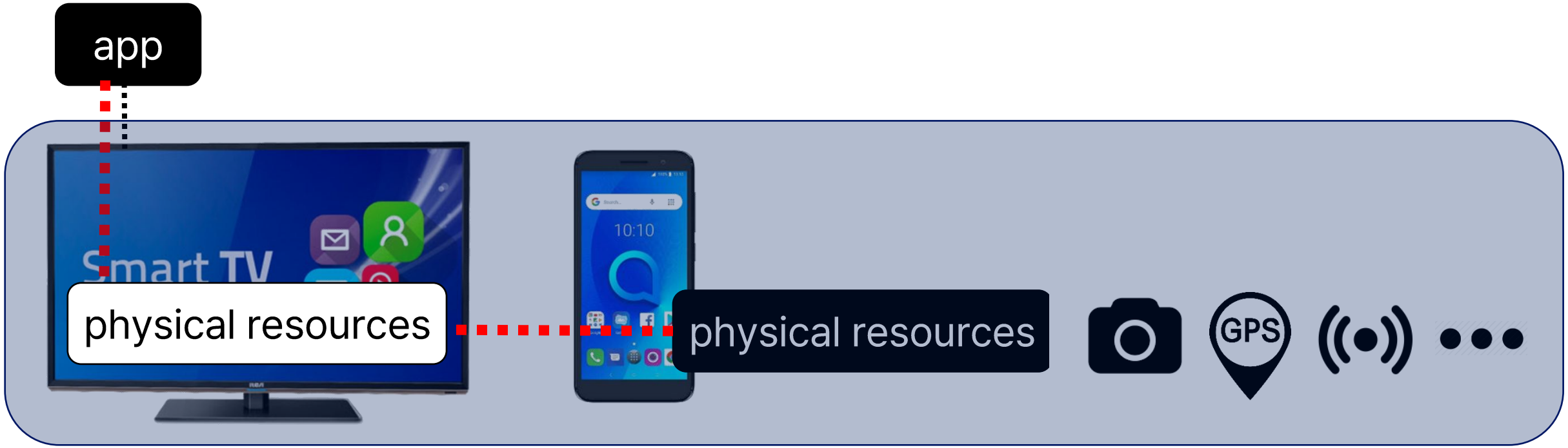
Multi-device Computing Model

- **Transparent cross-device resource sharing**
  - allows an app executing on one device to use resources on another device transparently



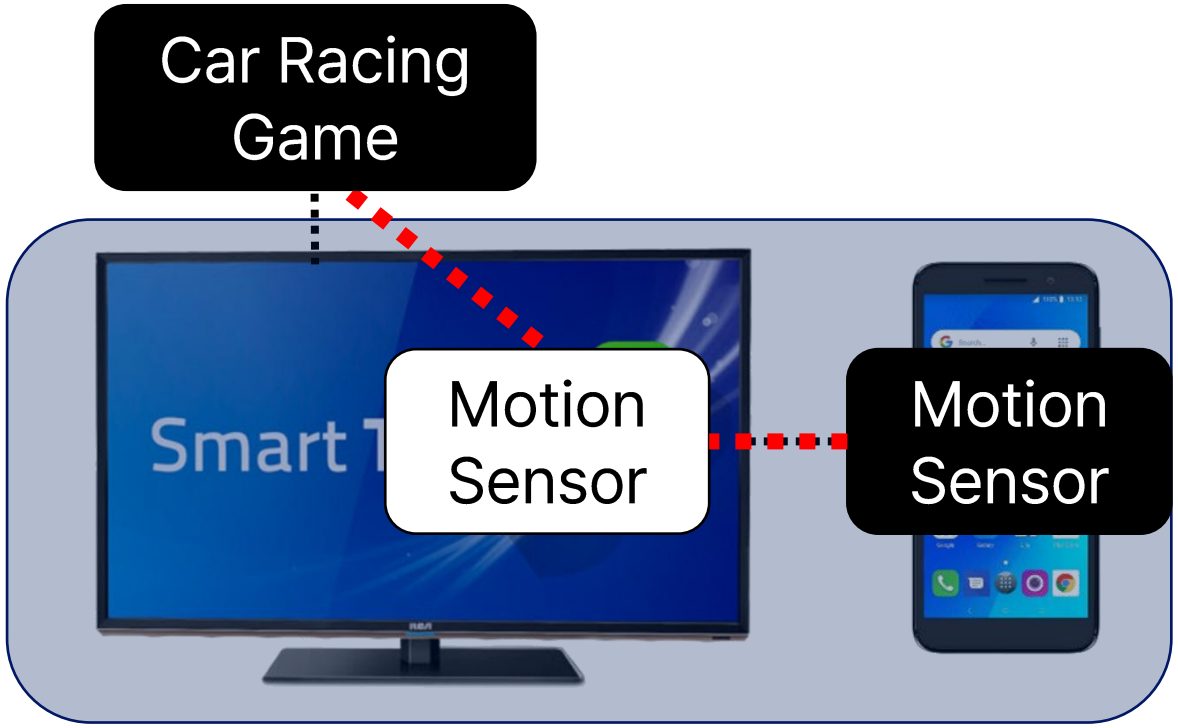
Multi-device Computing Model

- **Transparent cross-device resource sharing**
  - allows an app executing on one device to use resources on another device transparently
    - physical resources: camera, sensors, etc (**RIO, MobiSys '14**)



Multi-device Computing Model

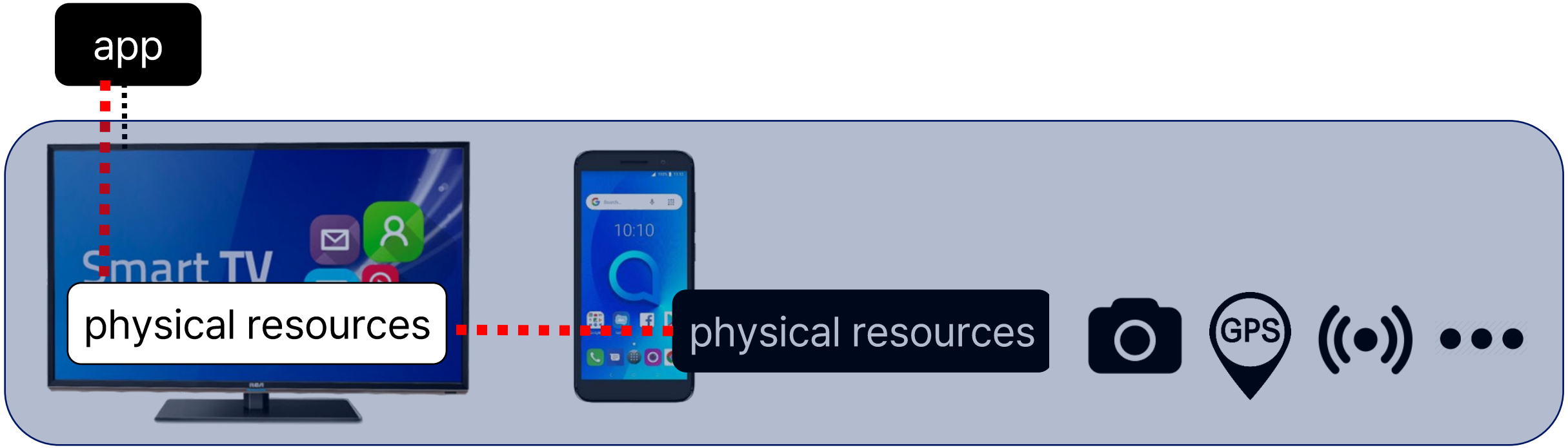
- **Transparent cross-device resource sharing**
  - allows a user to use resources across different devices transparently
    - physical resources: camera, sensors, etc (**RIO, MobiSys '14**)
    - logical (app) services: login, payment, browsing, PDF viewer, etc (**M+, MobiSys '17**)



RIO & M+

Multi-device Computing Model

- **Transparent cross-device resource sharing**
  - allows an app executing on one device to use resources on another device transparently
    - physical resources: camera, sensors, etc (**RIO, MobiSys '14**)



Any other ideas for

new abstractions (computing models)

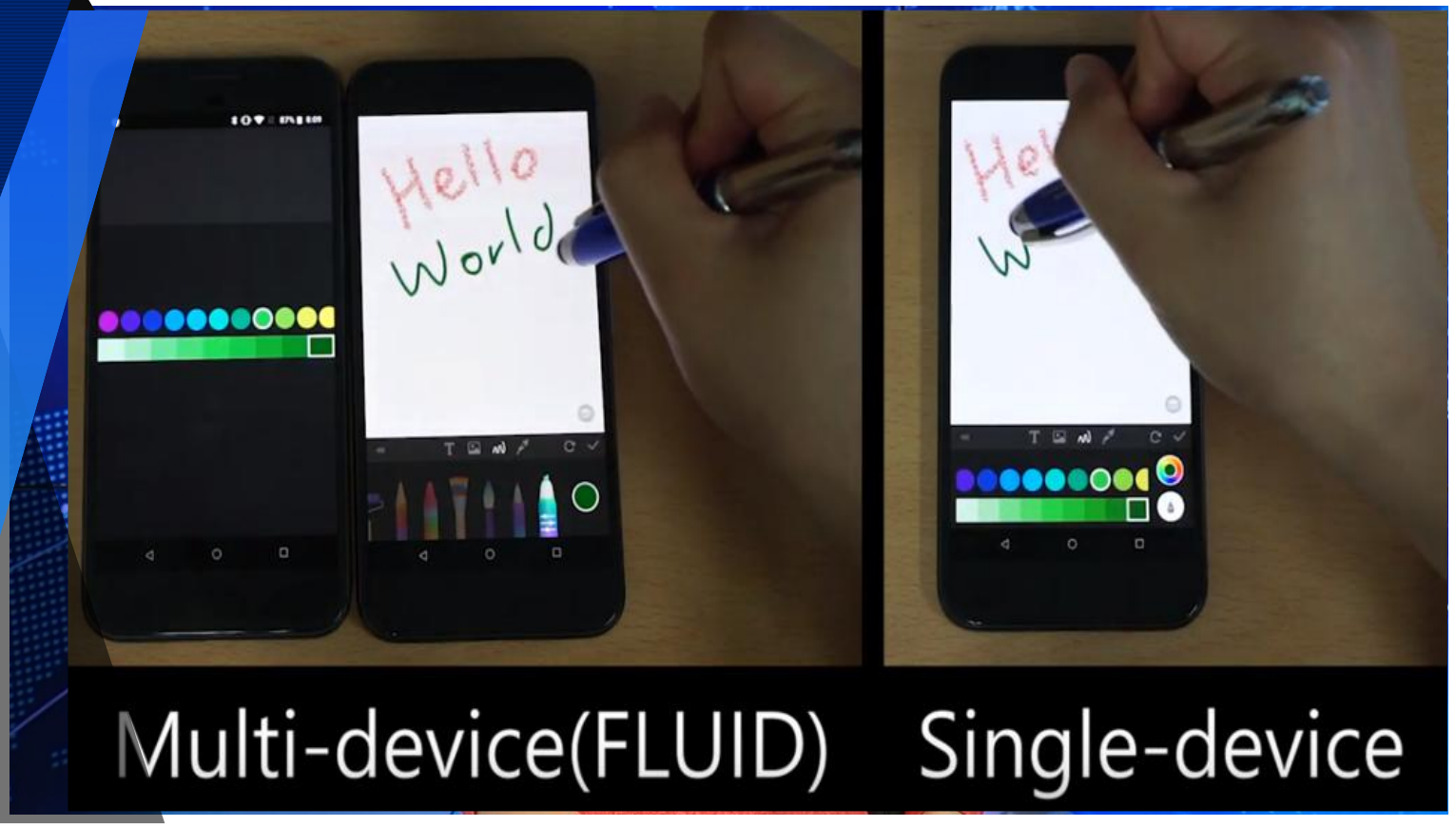
for multi-device experience (environments)?

FLUID

toward

New Multi-device

Computing Model

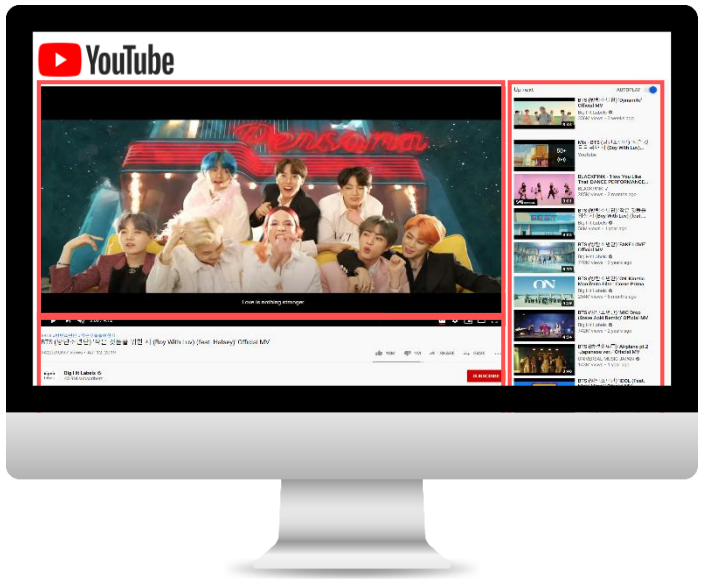


What does FLUID do?

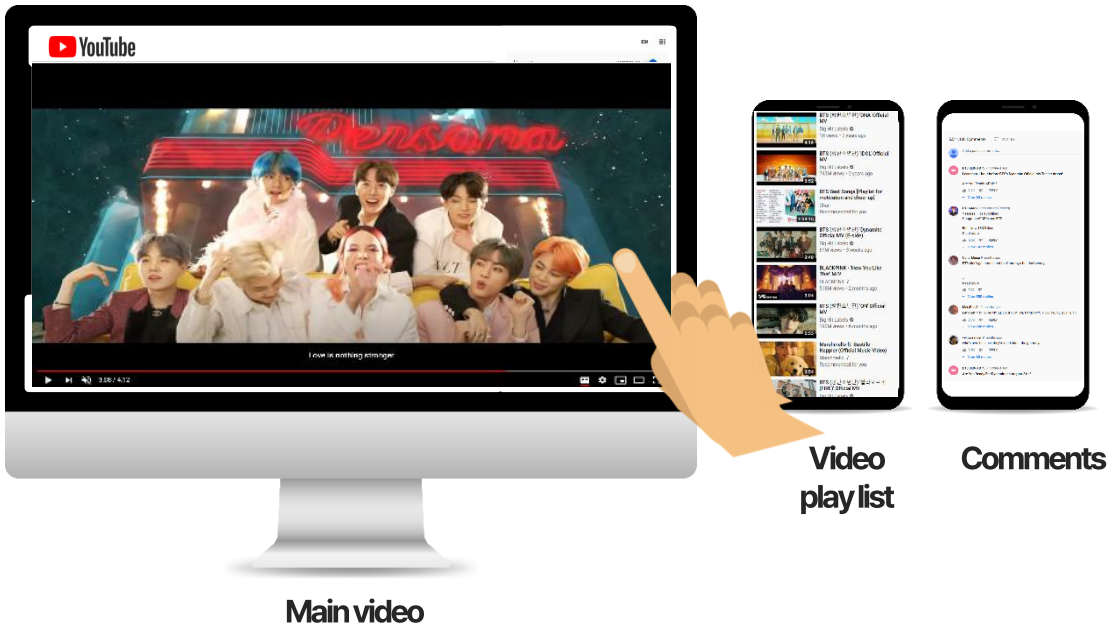
- it partitions an app at the unit of UI, distributes UIs across different devices, and runs each UI simultaneously on different devices

Without Code Modification !!

As-is Single-device (single-device one-app) Paradigm

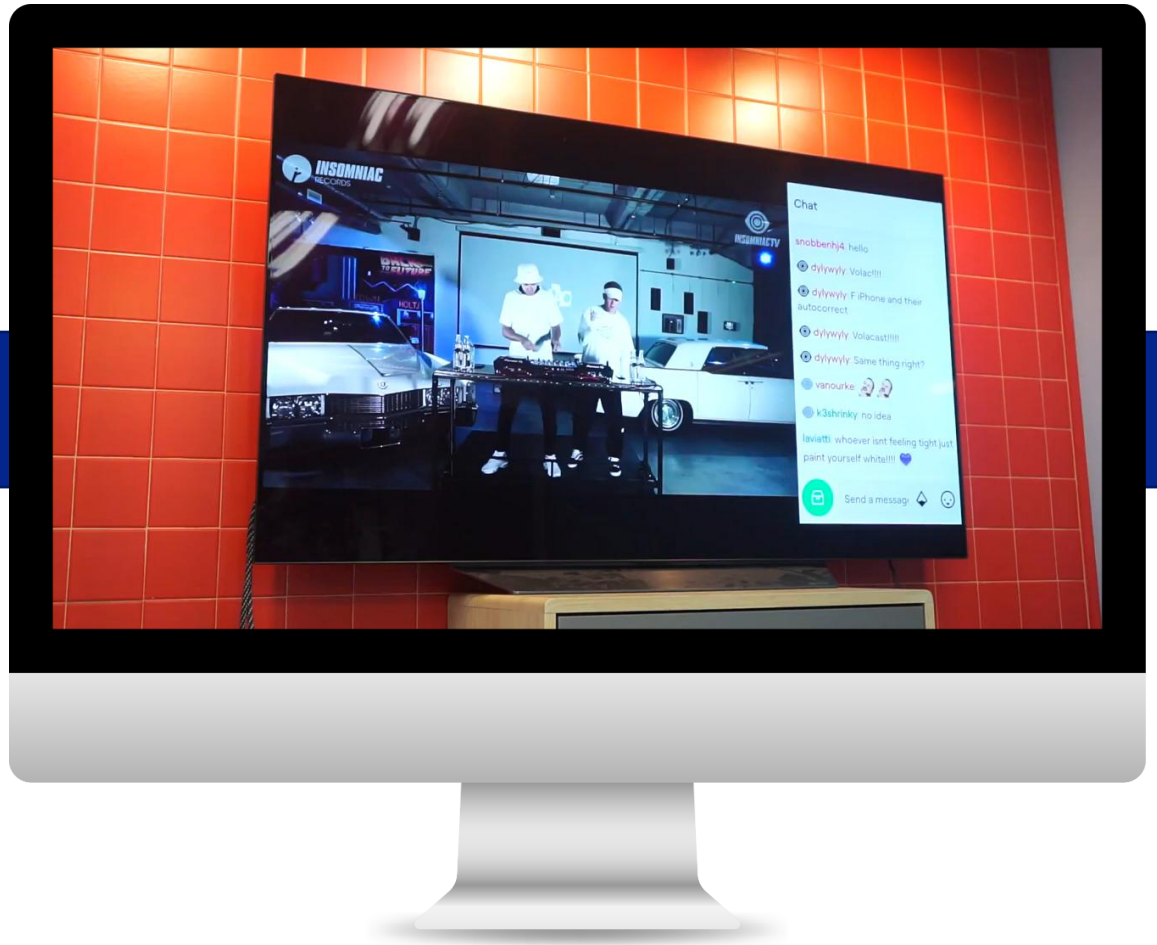


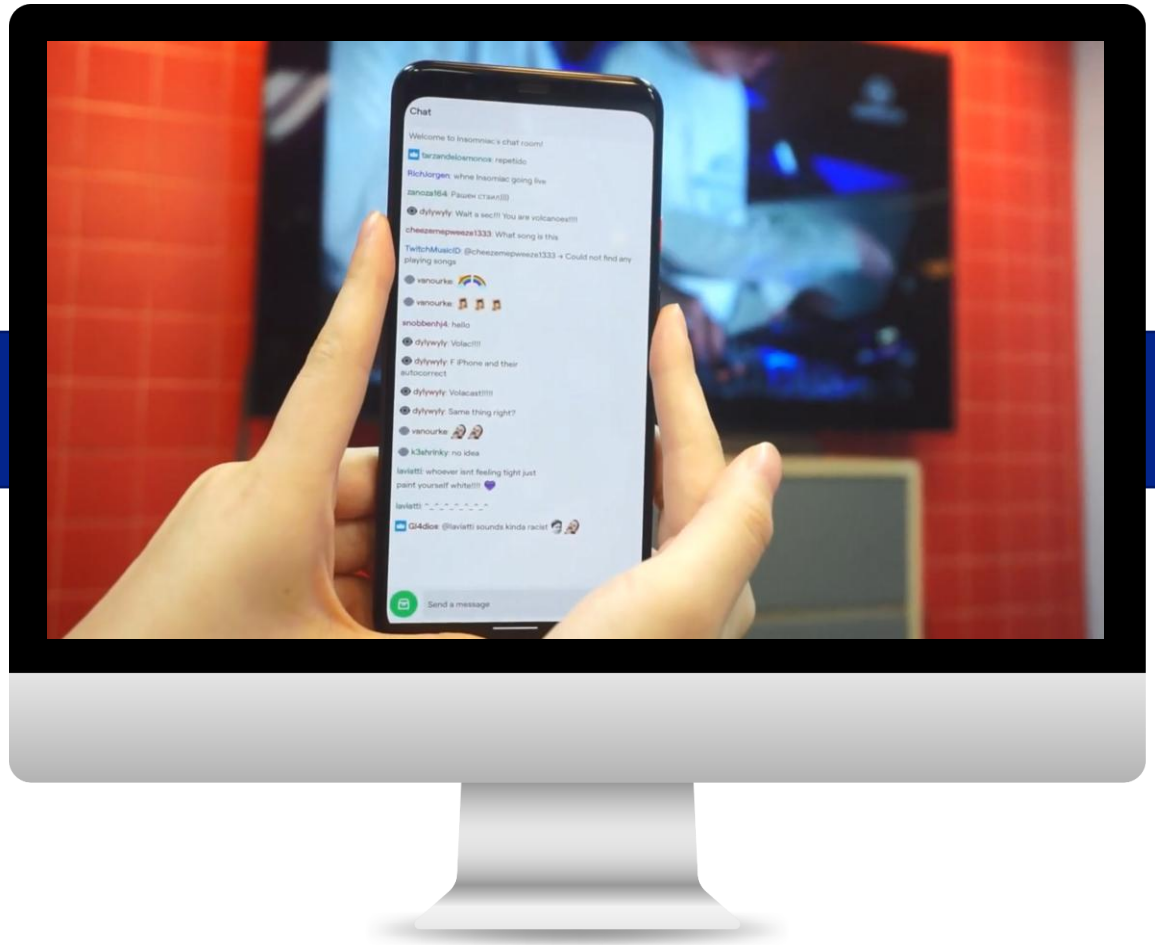
FLUID Multi-device (multi-device one-app) Paradigm



Through FLUID's UI distribution technology, we provide a new level of multi-device UX that has been not possible in a single-device environment

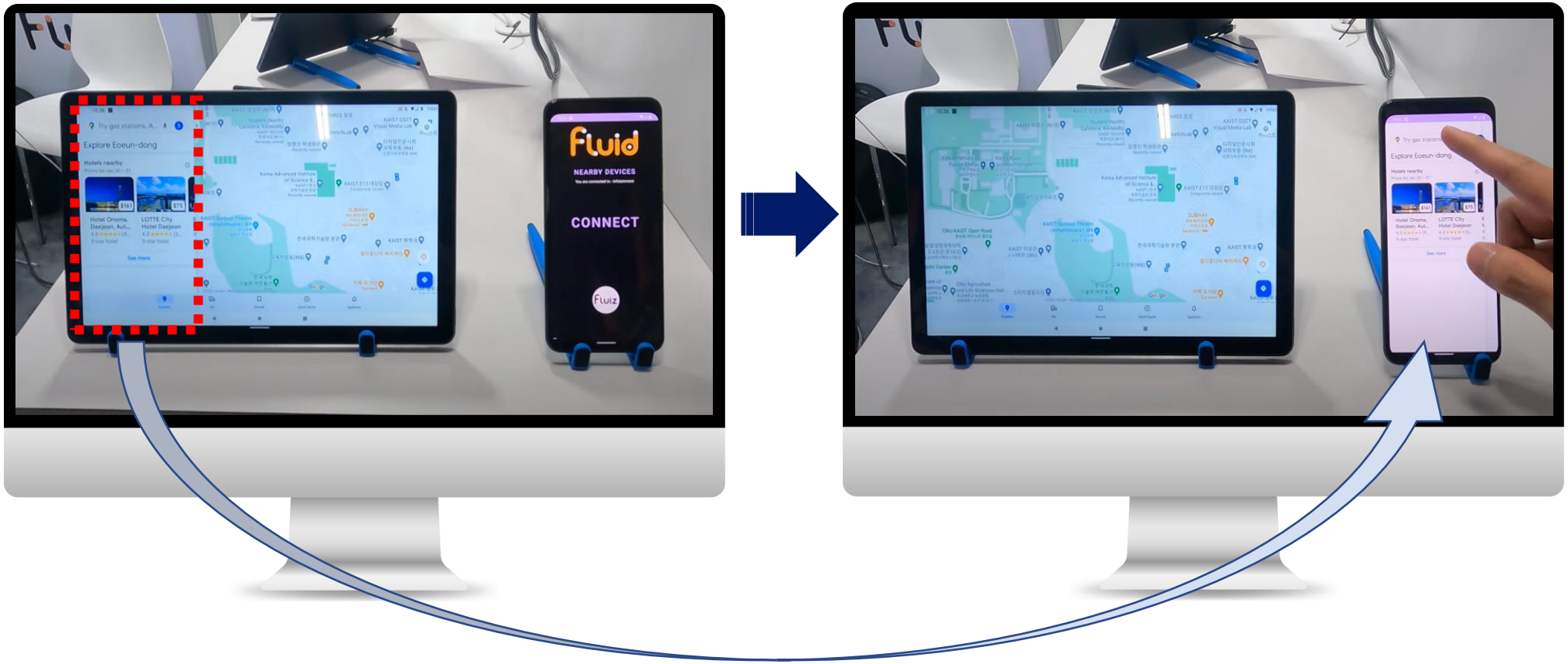
Example #2
Distributing Twitch app's live chat window (Video resized to full-screen)





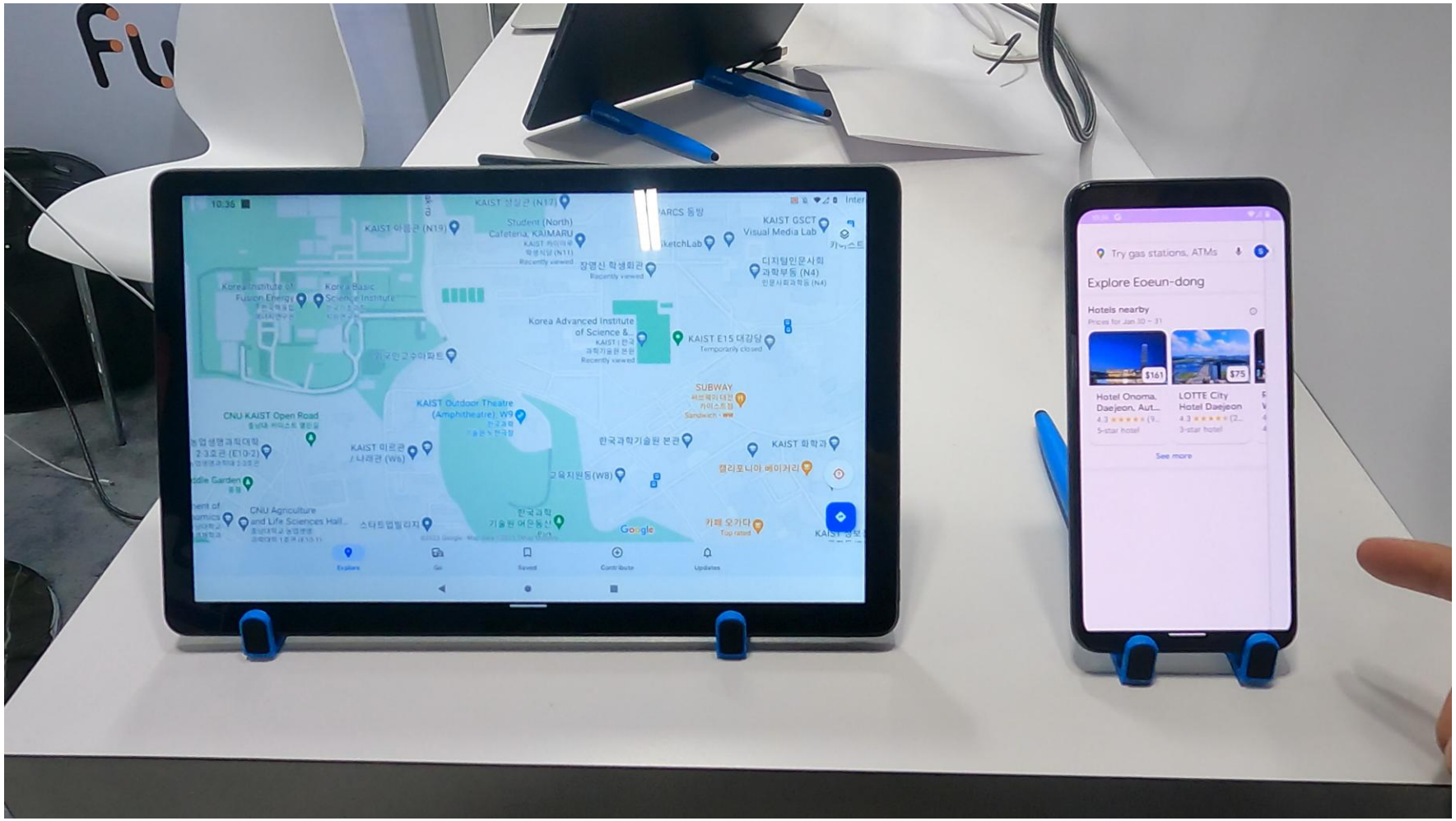
Example # 3

G o o g l e M a p s (Video resized to full-screen)



Example # 3

G o o g l e M a p s (Video resized to full-screen)



Existing solutions

- **Customized apps**
  - Extra engineering efforts
  - Low applicability

Google docs

Netflix

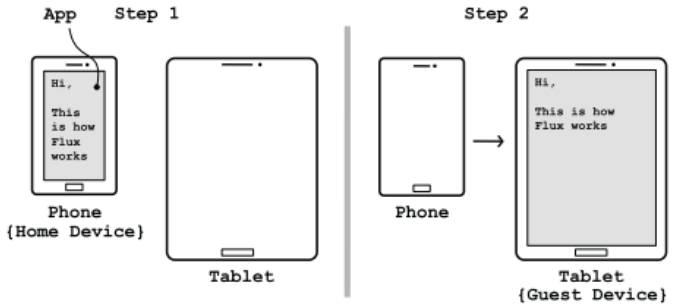
Smartwatch apps

- **Screen mirroring**
  - Low flexibility
    - Supports only full screen
  - Low responsiveness
    - for high resolutions

Chromecast

Vysor

- **App migration**
  - Low flexibility
    - Supports only full screen
    - Cannot support concurrent usage



Flux [EuroSys'15]

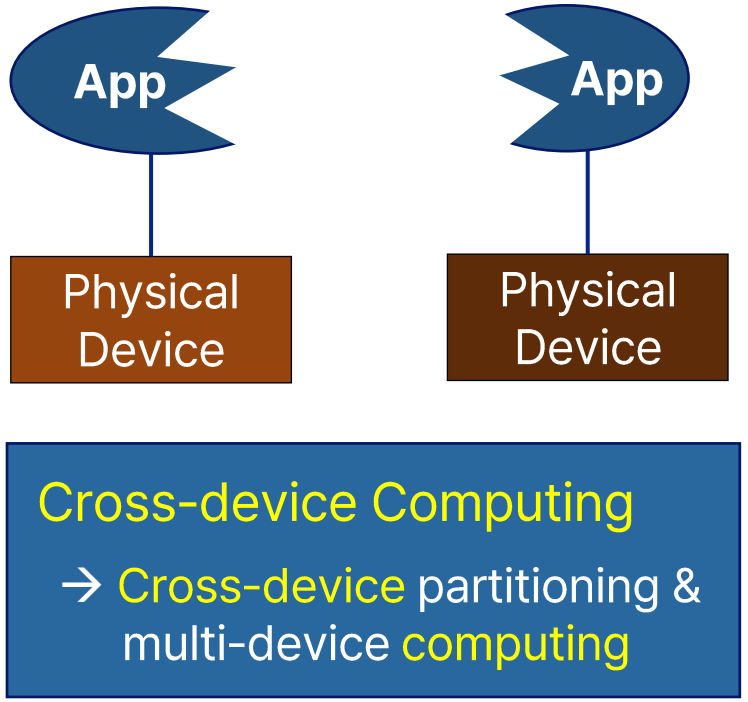
Goal

- Design a new mobile platform that supports multi-device computing

  by distributing UI objects to different devices

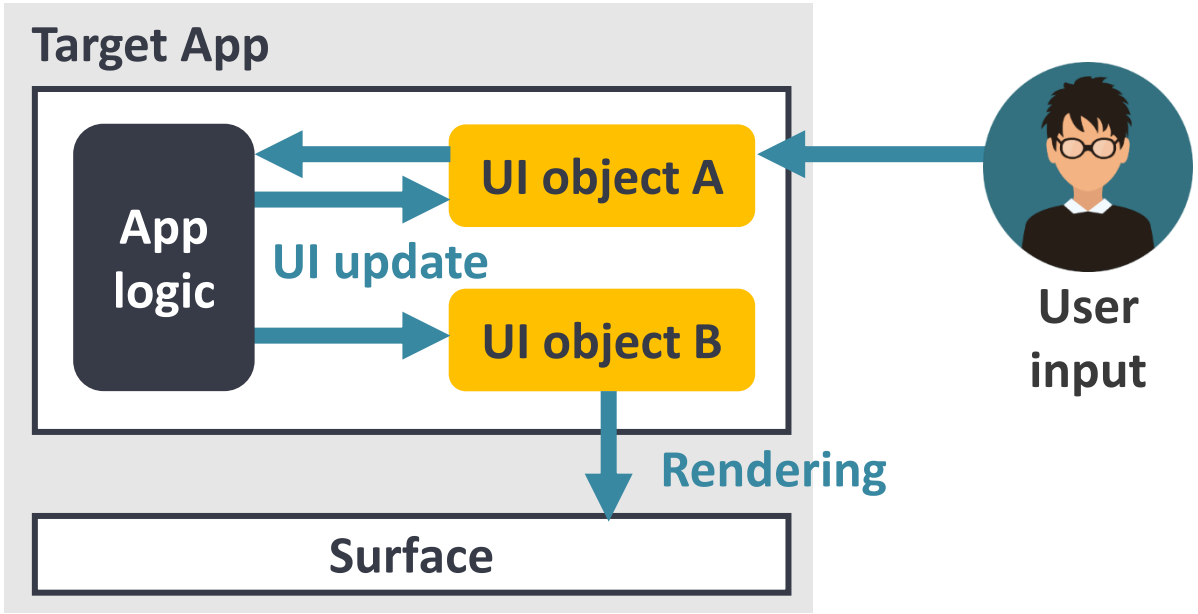
  in a **flexible**, **transparent** and **responsive** manner

FLUID
(FLexible UI Distribution)



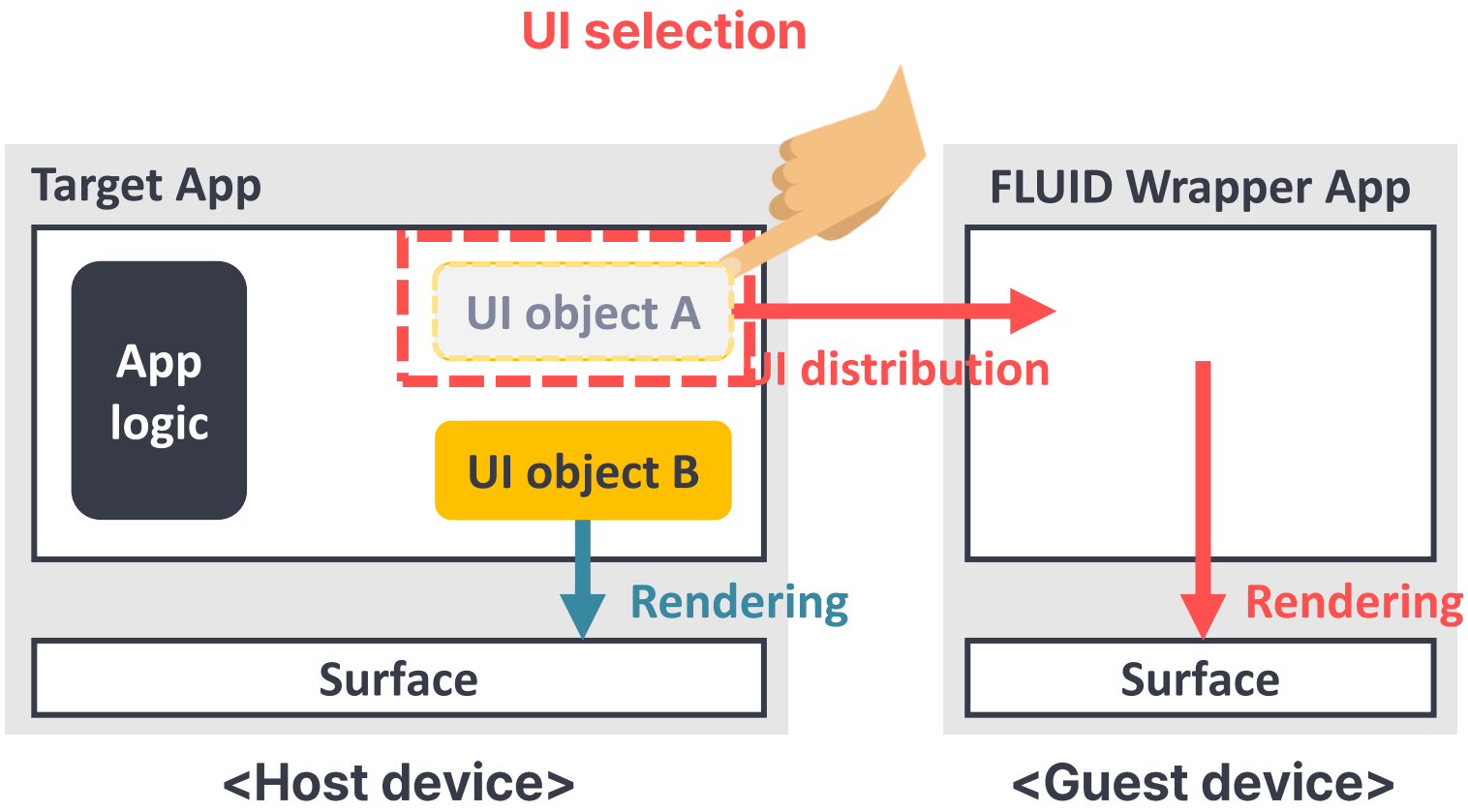
FLUID overview

- **Key idea**: separation between app logic & UI parts



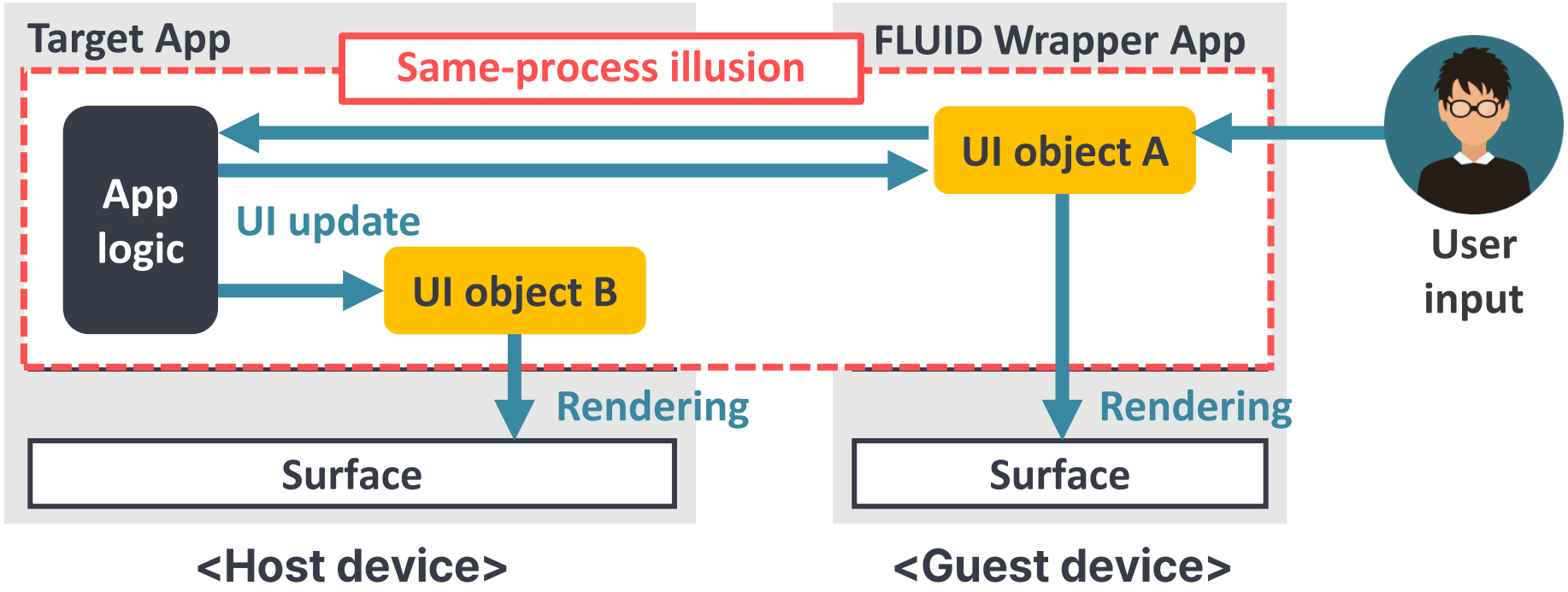
FLUID overview

- **Key idea**: separation between app logic & UI parts
  1) Distributing target UI objects to remote devices and rendering them



FLUID overview

- **Key idea**: separation between app logic & UI parts
  1) Distributing target UI objects to remote devices and rendering them
  2) Giving an illusion as if app logic and UI objects were in the same process by extending ***intra-app interaction*** to multi-device environments



Why is FLUID good?

- **Flexibility**
  - Allow users to interact with multiple devices, via fine-grained UI distribution, with maximum flexibility

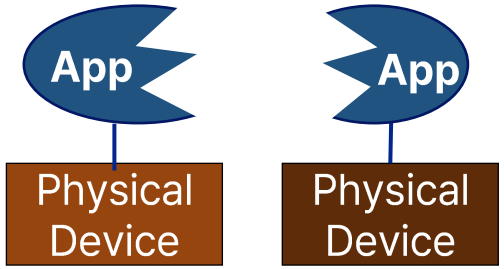
  (UI is the minimum unit of functionality from user's viewpoint)

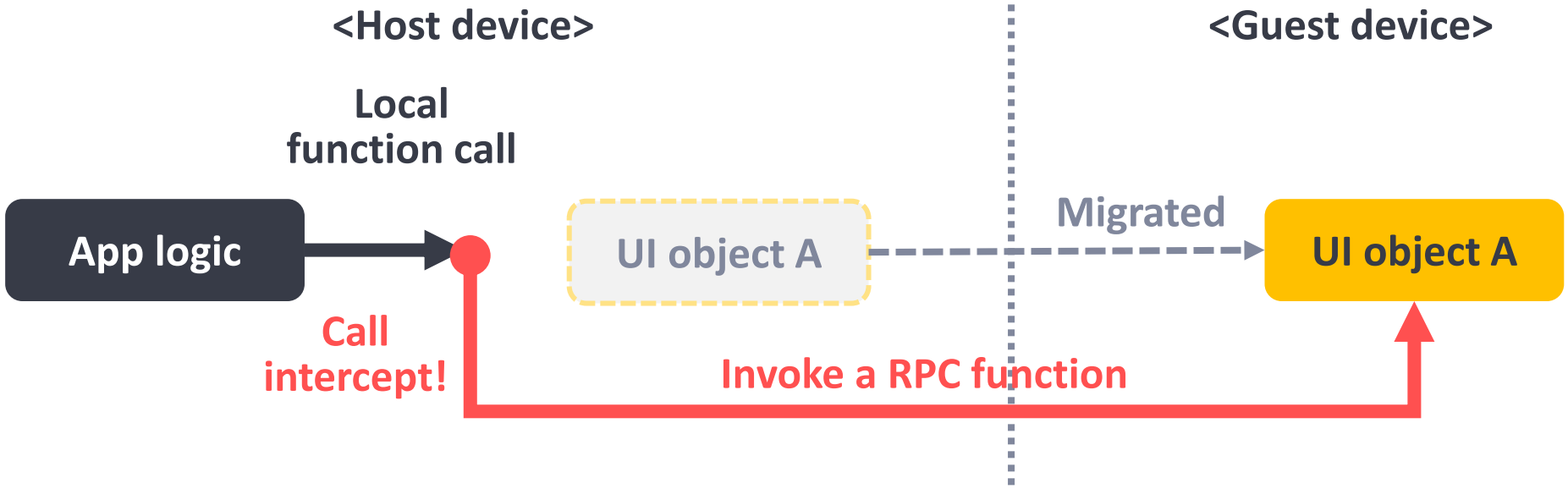
- **Transparency**
  - Support legacy apps without any modification to them
  - Develop new multi-surface apps under the existing programming model

- **Responsiveness**
  - Require less network transmission when moving UIs instead of full screen

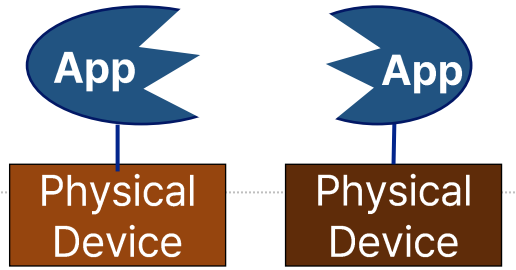
Problems

- **P1**. How to maintain interaction between app logic & UI objects?
  - Such interaction is achieved via local function calls
    - e.g., *TextView.setText()*, *ImageView.setImageResource()*, etc.
  - However, local functions cannot be executed across devices
  - **Our solution**: transparent RPC transformation in *Android VM (ART)*





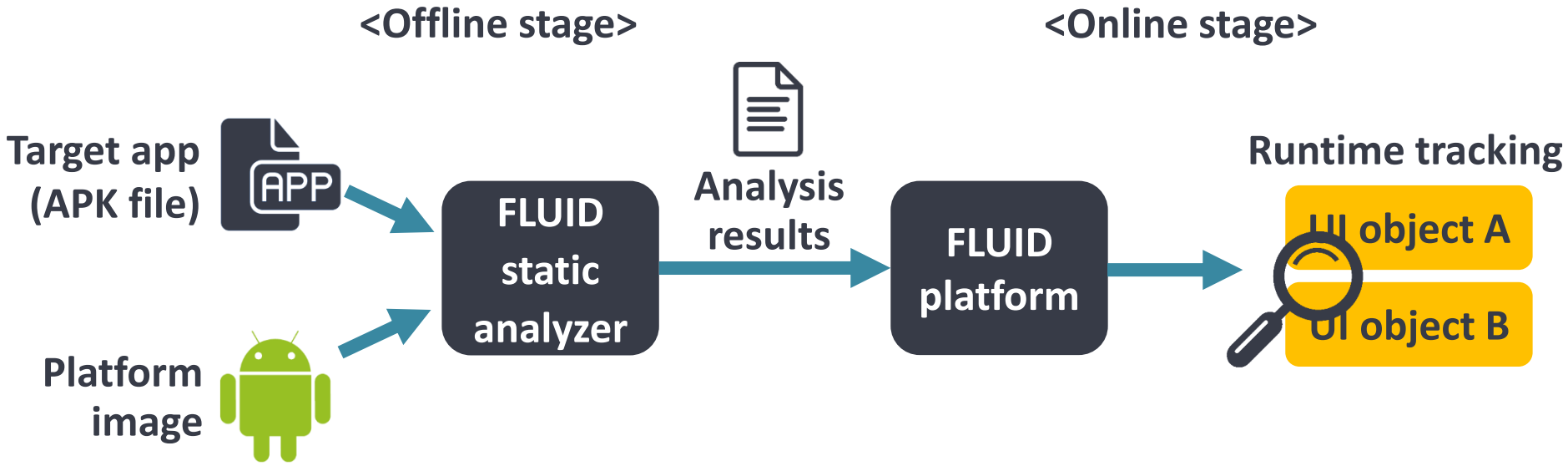
Problems



- **P2**. How to split & distribute UI objects?
  - We aim to minimize the amount of network transfer (graphical states) needed for UI rendering
    - To reduce network overhead
  - However, for some app-specific custom UIs, it is not easy to precisely determine which graphical states are used

Problems

- **P2**. How to split & distribute UI objects?
  - We aim to minimize the amount of network transfer (graphical states) needed for UI rendering
    - To reduce network overhead
  - However, for some app-specific custom UIs, it is not easy to precisely determine which graphical states are used
  - **Our solution**: Selective UI distribution



Evaluation environment

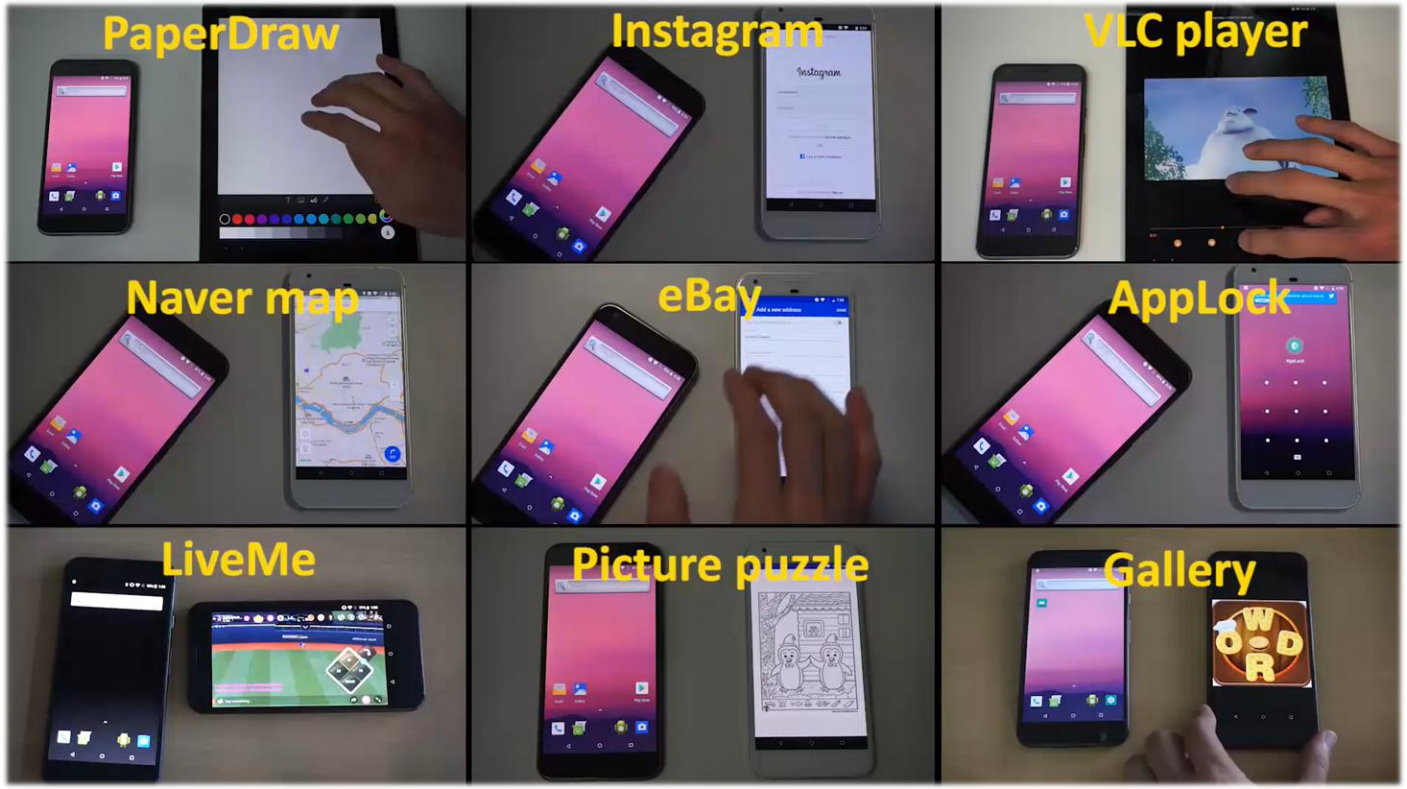
- Used Google Pixel XL (smartphone) & Pixel C (tablet)
  - Phone-to-phone
  - Phone-to-tablet
  - Tablet-to-phone
- On the Same WiFi network

Pixel XL

Pixel C

App coverage

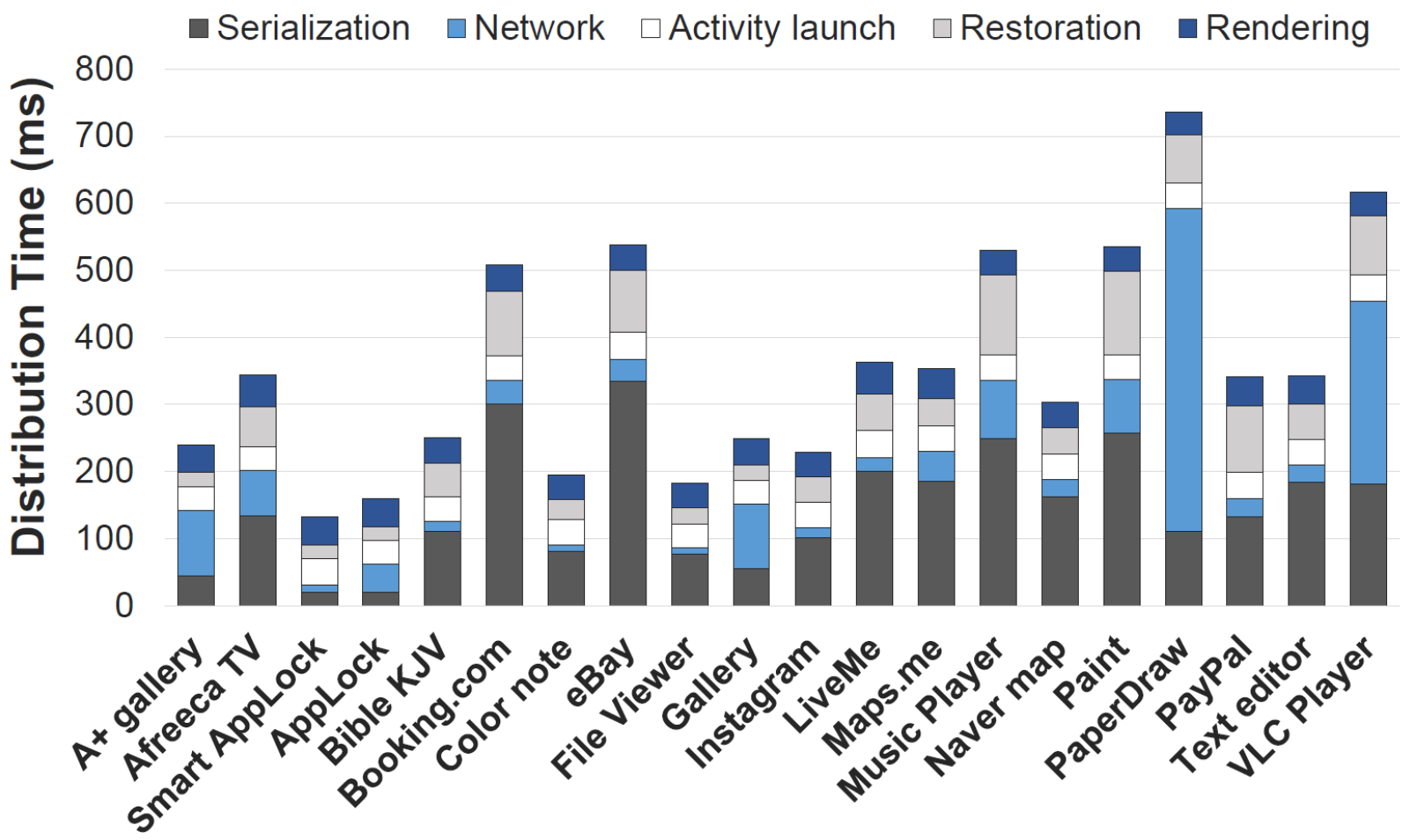
- Using 20 legacy apps for 10 multi-device scenarios
  - All legacy apps use their own custom UIs
- FLUID supports various legacy apps successfully



Use case scenario	UI type	App name
Login with personal device	Editor	Instagram
		Paypal
Fill in information collaboratively	Text, editor	eBay
		Booking.com
Chatting with different device while broadcasting	Button, editor	LiveMe
		Afreeca TV
Search destination with different device	Button, editor	Naver map
		Maps.me
Control media with different device	Seek bar, button	VLC Player
		Music Player
Control painting tool with different device	Scroll, image, button	PaperDraw
		Paint
Sharing photo to public device	Image	Gallery
		A+ Gallery
Unlock pattern with personal device	Pattern lock	Smart app lock
		AppLock
Read document with different device	Text, scroll	File Viewer
		Bible KJV
Edit text on different device	Editor	Color note
		Text editor

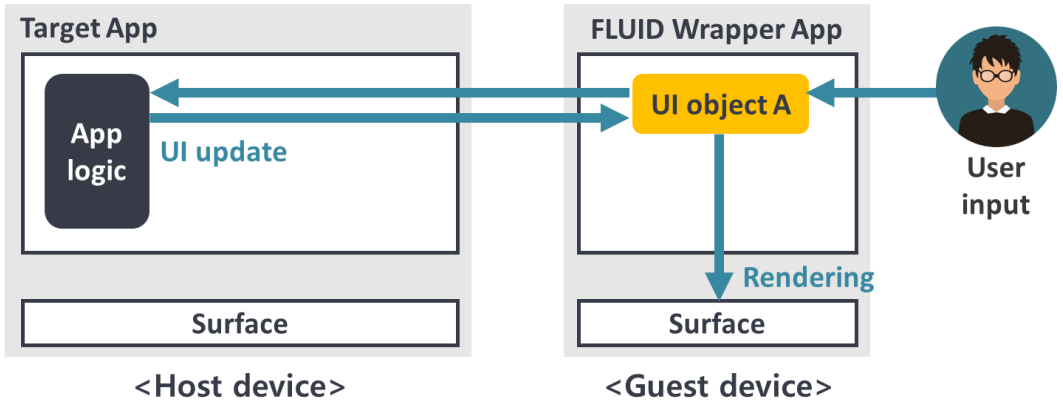
UI distribution time

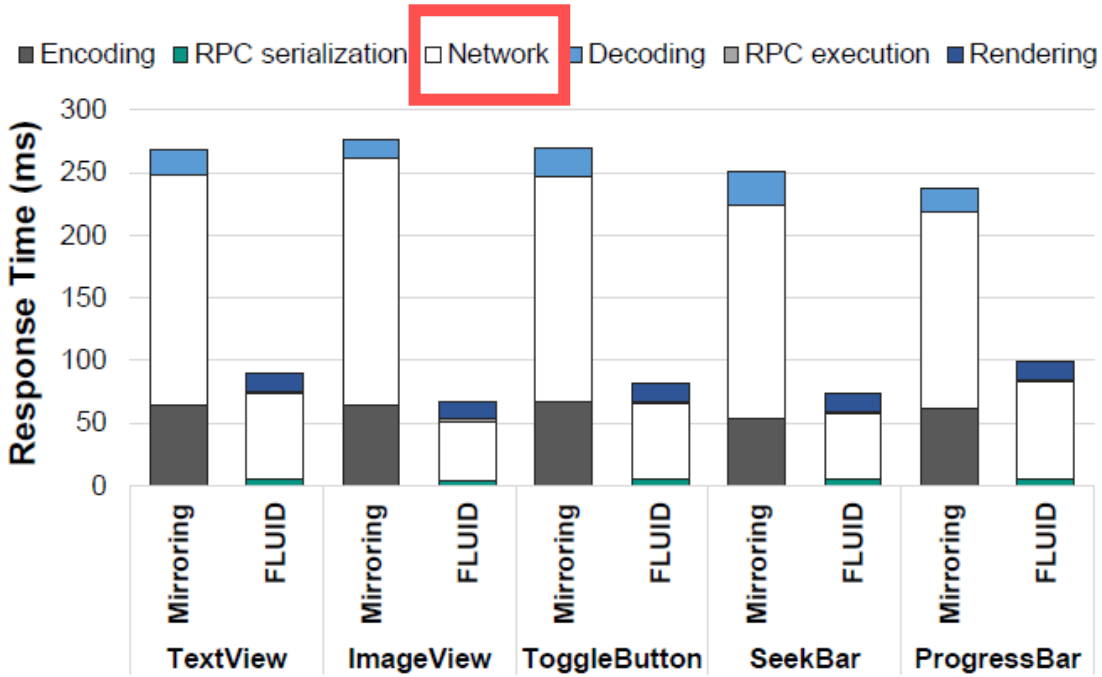
- It ranges from 132 to 735ms ➔ Fast enough for interactive use



UI response time

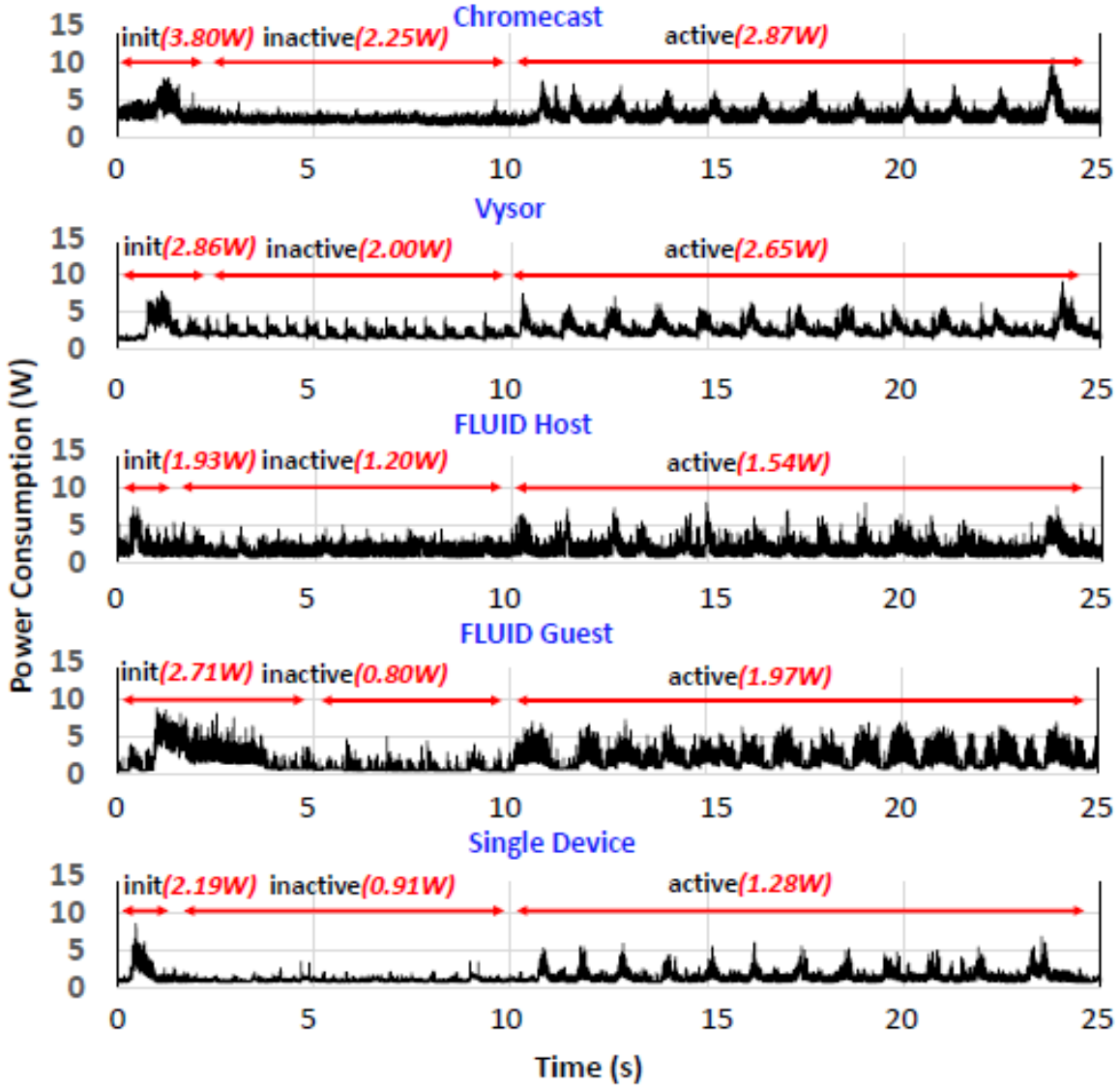
- FLUID outperforms SCRCPY by 2x to 4x
  - SCRCPY is an open-source screen mirroring app
- Network transfer time ➔ performance bottleneck





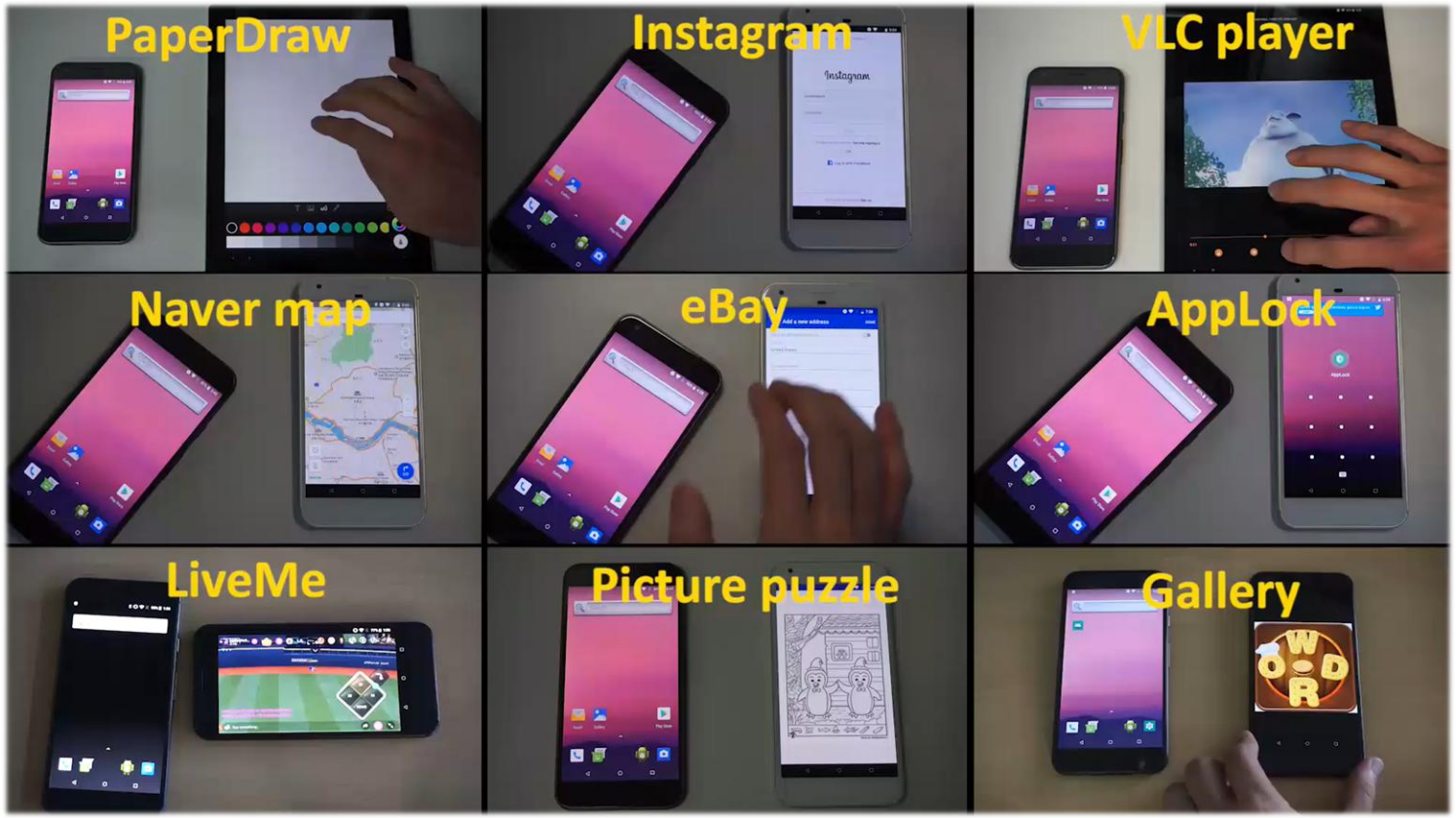
Power consumption

- Measured by Monsoon power monitor
- Power consumption follows the same trend as network transfer over time
- Efficient use of wireless network ➔ reducing power consumption
  - FLUID consumed up to 46% less power than chromecast & Vysor



Summary

- Designed **FLUID** that supports new multi-surface interaction
  - Separation between app logic & UIs
- Evaluated with 20 legacy apps for 10 multi-surface scenarios



References

- “A-Mash: Providing Single-App Illusion for Multi-App Use through User-centric UI Mashup,” ACM MobiCom 2022
- “**FLUID-XP: Flexible User Interface Distribution for Cross-Platform Experience”, ACM MobiCom 2021**
- **"FLUID: Flexible User Interface Distribution for Ubiquitous Multi-device Interaction“, ACM MobiCom 2019**
- "Mobile Plus: Multi-device Mobile Platform for Cross-device Functionality Sharing”, ACM MobiSys 201

Best Paper Award